Algoritmi paraleli si distribuiți

Tipuri și nivele de paralelism

+POSIX

Tipuri de procesări paralele

Taxonomia lui Flynn:

- SISD – **S**ingle **I**nstruction stream, **S**ingle **D**ata stream
- SIMD - **S**ingle **I**nstruction stream, **M**ultiple **D**ata stream
- MISD - **M**ultiple **I**nstruction stream, **S**ingle **D**ata stream
- MIMD - **M**ultiple **I**nstruction stream, **M**ultiple **D**ata stream

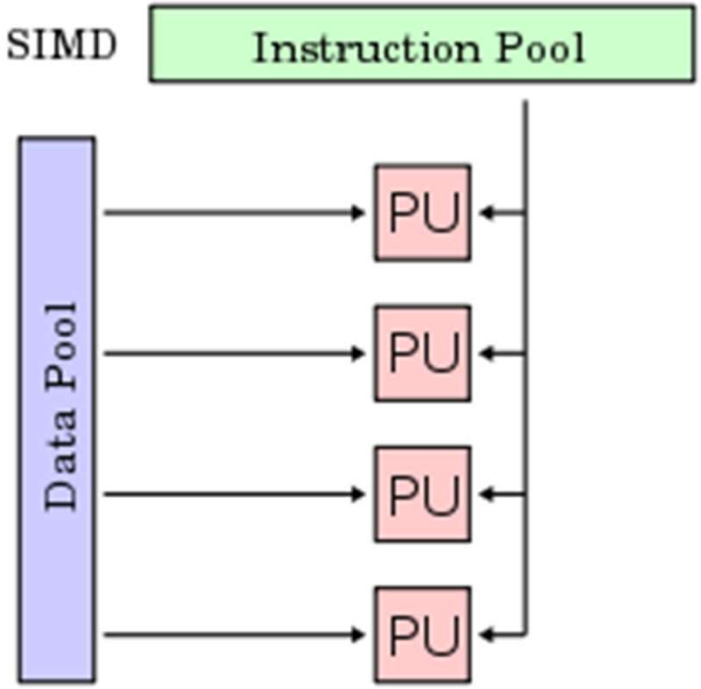
Flynn, M. J. (1972). Some computer organizations and their effectiveness. *IEEE transactions on computers*, *100*(9), 948-960.

SISD – Single Instruction stream, Single Data stream

- Un singur procesor, intr-un singur sistem care are o singură memorie.
- Instrucțiunile se execută secvențial.
- Procesarea paralelă se poate realiza numai cu ajutorul a **mai multor unități funcționale** (i.e., mai multe sisteme) si/sau prin **procesare pipeline**.

SIMD – Single Instruction stream, Multiple Data stream

- Reprezintă un sistem cu mai multe procesoare care se află sub comanda unei unități de control comune.
- Toate procesoarele primesc/execută aceleași instrucțiuni, dar pe date diferite.



SIMD

- Avantaje
  - Mai puțin HW necesar față de MIMD, este nevoie doar de o unitate de control (care poate avea mai multe procesoare).
  - Este nenesară mai puțină memorie deoarece este nevoie doar de o singură copie a programului.
  - Comunicare minimală între procesoare.
  - Ușor de înțeles.
- Dezavantaje
  - Infrastructură rigidă care se pretează numai pentru problemele bine structurate.

SIMD - exemplu

- Sistemele SIMD sunt eficiente pentru programele realizează aceeasi procesare pe multe date în paralel.
- Ex. 1. Procesorul k execută c[k] = a[k]+b[k]

```
for (i=0; i<1000; i++)
        c[i] = a[i]+b[i];
```

- Ex. 2. Procesoare distincte **nu** pot executa instrucțiuni distincte în același ciclu processor.

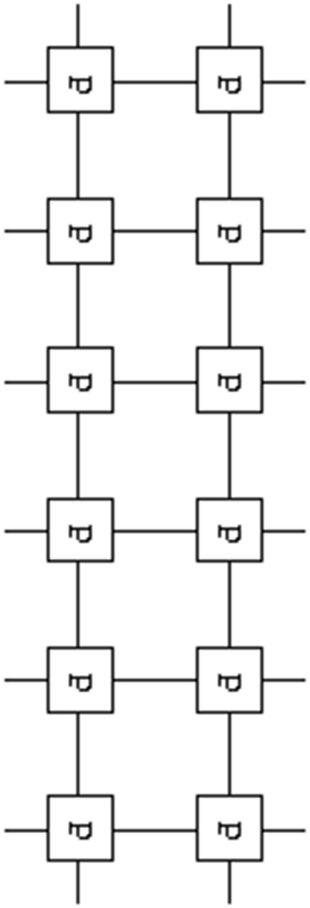
```
for (i=0; i<10; i++)
        if (a[i]<b[i])
                c[i] = a[i]+b[i];
        else
                c[i] = 0;
```

MISD –Multiple Instruction stream, Single Data stream

- Procesoarele primesc/execută instrucțiuni diferite, dar lucrează pe aceleași date.
- **Acest model este teoretic!**

MIMD – Multiple Instruction stream, Multiple Data stream

- Sistemul este capabil să ruleze mai multe programe în același timp.
- Cele mai multe sisteme multiprocesor aparțin acestei categorii.
- Fiecare procesor poate executa un flux separat de instrucțiuni asupra unor date proprii pe care le gestionează local.
- Acest model este cu memorie distribuită deoarece memoria este distribuită între procesoare în loc să fie partajată (i.e., plasată central si accesibilă tuturor procesoarelor).



MIMD

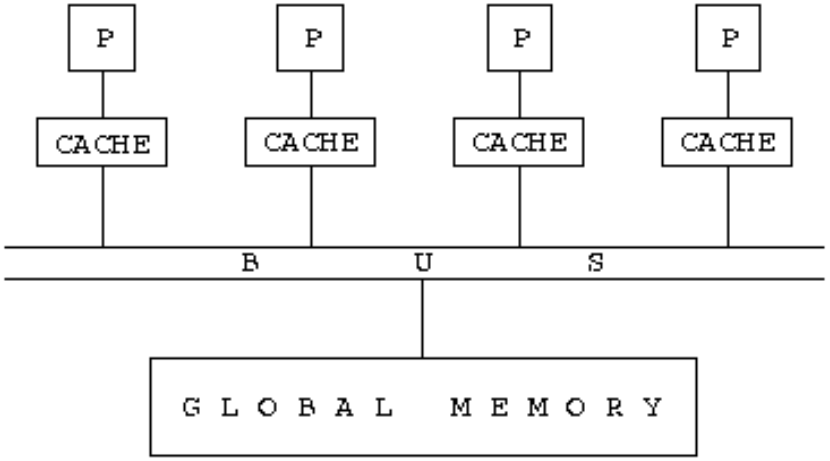
- **Avantaje**:
  - Poate fi ușor/rapid/ieftin construit pe baza unor microprocesoare existente.
  - Flexibilitatea: potrivit pentru probleme care nu se inscriu în anumite șabloane.
  - Poate utiliza alt HW pentru a obține sincronizare rapidă, ceea ce poate face ca sismemu MIMD sa opereze in modul SIMD.
- **Dezavantaje**
  - Complexitate crescută (fiecare procesor are propria unitate de control).
  - Necesită mai multe resurse (duplicarea programului, OS, etc.).
  - Proiectarea/depanarea mai dificilă a programelor.

PRAM: Parallel Random Access Machine

- Sistemul este multiprocesor dar cu memorie partajată.
- Toate procesoarele au acces la o memorie comună care este partajată între ele cu ajutorul unei magistrale de date (en., bus network).
- Modelul ***ideal*** PRAM este folosit în studiile teoretice ale algoritmilor paraleli. Acest model permite accesarea memoriei de către fiecare procesor cu aceeași viteză (i.e., în aceeași cantitate de timp).
- Este omologul modelului von Neuman pentru sistemele secvențiale.

PRAM: Parallel Random Access Machine (cont.)

- Este compus din:
  - **p** rocesoare, care rulează sincron aceleași instrucțiuni.
  - Fiecare procesor are o memorie locală.
  - Fiecare procesor este conectat la memoria partajată.
  - Accesul la memoria partajată se face în timp constant, intr-un pas.
- Limitarea modelului PRAM
  - Nu ia în calcul comunicarea și se concentrează doar pe sarcinile paralele.

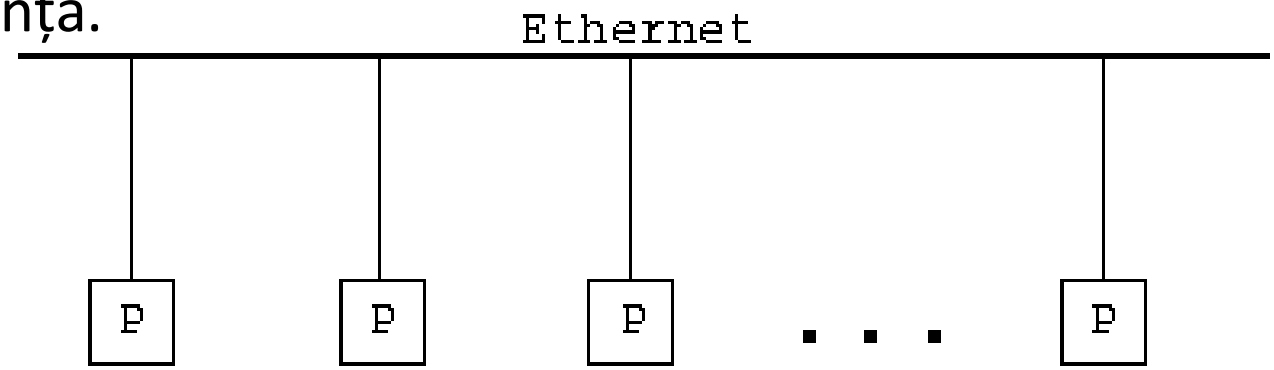


PRAM: Parallel Random Access Machine (cont.)

- In practică, scalarea sistemelor cu multiprocessor necesită introducerea/utilizarea unei arhitecturi ierarhice a memoriei.
- Frecvența cu care este accesată memoria partajată se poate reduce prin crearea de *chache*-uri
  - *Chache* = copie ale datelor frecvent folosite care se gasește de fiecare procesor.
  - ***Obs***: Accesarea cache-ului este mult mai rapidă decât accesarea memoriei partajate => așezarea/localizarea datelor este importantă!
  - Aspecte practice referitoare la crearea si administrarea unui cache:
    - Ce structura de date sa utilizam? Array, vector, heap, arbore, graf?
    - Care sunt cele mai dese operatii? Inserare, căutare, modificare, stergere?

Alte clase de sisteme

- .... Care se pot folosi uneori ca și sisteme paralele:
- LAN (Local Area Network): sistemele sunt fizic apropiate (ex., intr-o cladire) și sunt conectate intr-o rețea (rapidă!).
- WAN (Wide Area Network): sistemele sunt distribuite geografic la mare distanță.



Alte clasificări/taxonomii

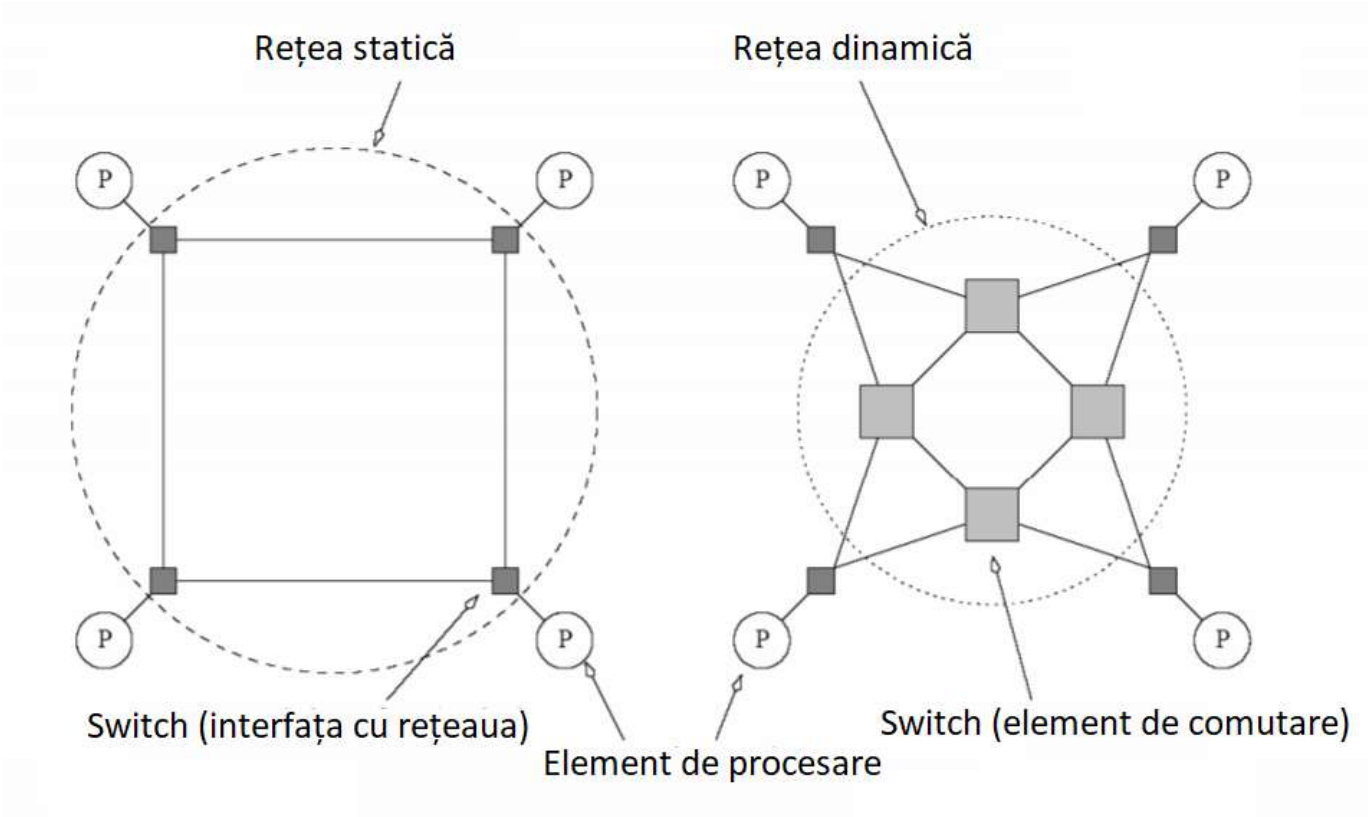
- Duncan
  - Propusa de Ralph Duncan in 1990.
  - Modifică taxonomia Flynn prin introducerea noțiunii de proces vectorial de tip pipeline (en., pipelined vector processes)
- Tse-yun Feng
  - Propune utilizarea noțiunii de *grad de paralelizare* pentru a clasifica diferitele arhitecturi ale claculatoarelor paralele.
  - Gradul de paralelizare poate fi maxim, mediu, minim.
  - Introduce noțiunea de utilzare a procesorului.

Organizarea fizică a sistemelor

Rețelele de interconectare

- Asigură conexiunile procesor-la-procesor și procesor-la-memorie
- Sunt clasificate în:
  - Statice
    - Constă dintr-un număr de legături punct-la-punct: rețea directă.
    - Sunt folosite pentru a conecta procesoarele între ele: sisteme cu memorie distribuită.
  - Dinamice
    - Rețea indirectă: rețeaua constă din elemente la care se atașează procesoarele.
    - Sunt folosite pentru a conecta procesoarele cu memoria: sisteme cu memorie partajată.

Rețea de conectare statică/dinamică



Metrici de evaluare pentru rețelele de interconectare

- Diametrul [*cu cât mai mic cu atât mai bine*]
  - Lungimea maximă dintre oricare două noduri.
- Conectivitatea [*cu cât mai mare cu atât mai bine*]
  - Numărul minim de conexiuni care trebuie distruse pentru a obține două componente conexe din rețeaua inițială.
  - Determină numărul de căi multiple.
- Lățimea bisecțiunii (en., bisection width) [*cu cât mai mare cu atât mai bine*]
  - Numărul minim de conexiuni care trebuie distruse pentru a obține două componente conexe de dimensiune egală.

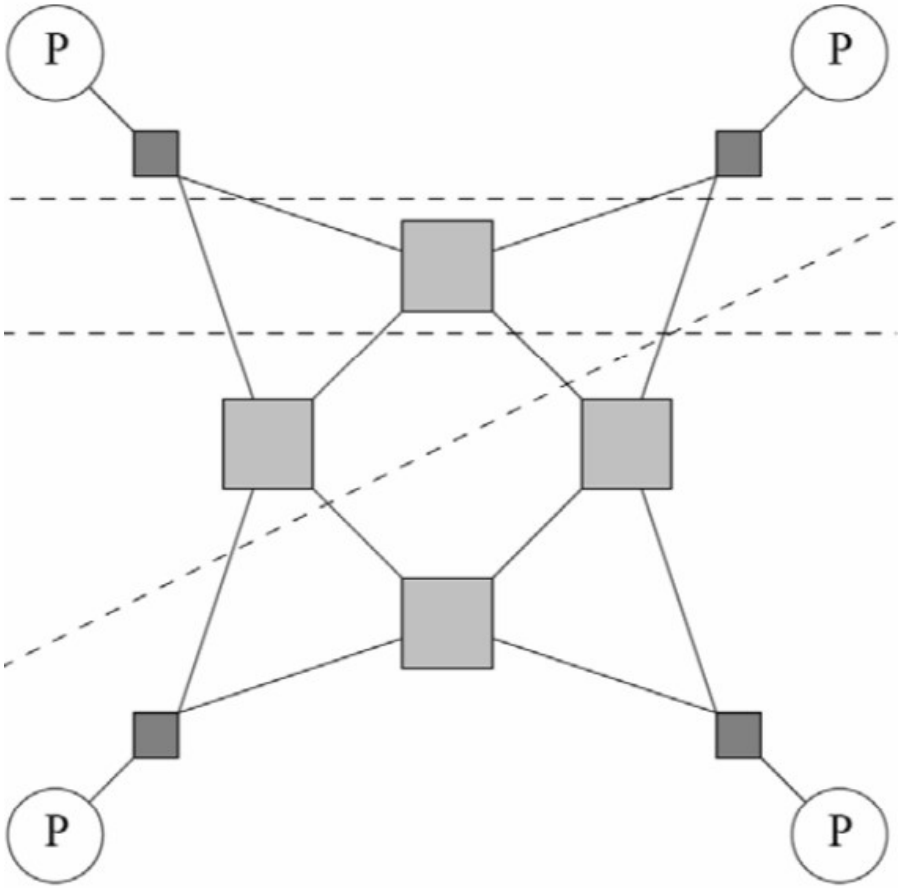
Metrici de evaluare pentru rețelele de interconectare (cont.)

- Lățimea de bandă a bisecțiunii (en., bisection band width) [*cu cât mai mare cu atât mai bine*]
  - Se aplică pentru rețele ponderate (en., weighted). Ponderile corespund lățimii de bandă, adică reprezintă cantitatea de date care poate fi transferată prin acea legătură/muchie.
  - Volumul minim de date care pot fi transferate între oricare două jumătăți ale rețelei.
- Costul [*cu cât mai mic cu atât mai bine*]
  - Numărul de legături (i.e., arce) din rețea.

- *Care algoritmi de grafuri adresează aceste probleme?*

Metrici în rețelele dinamice

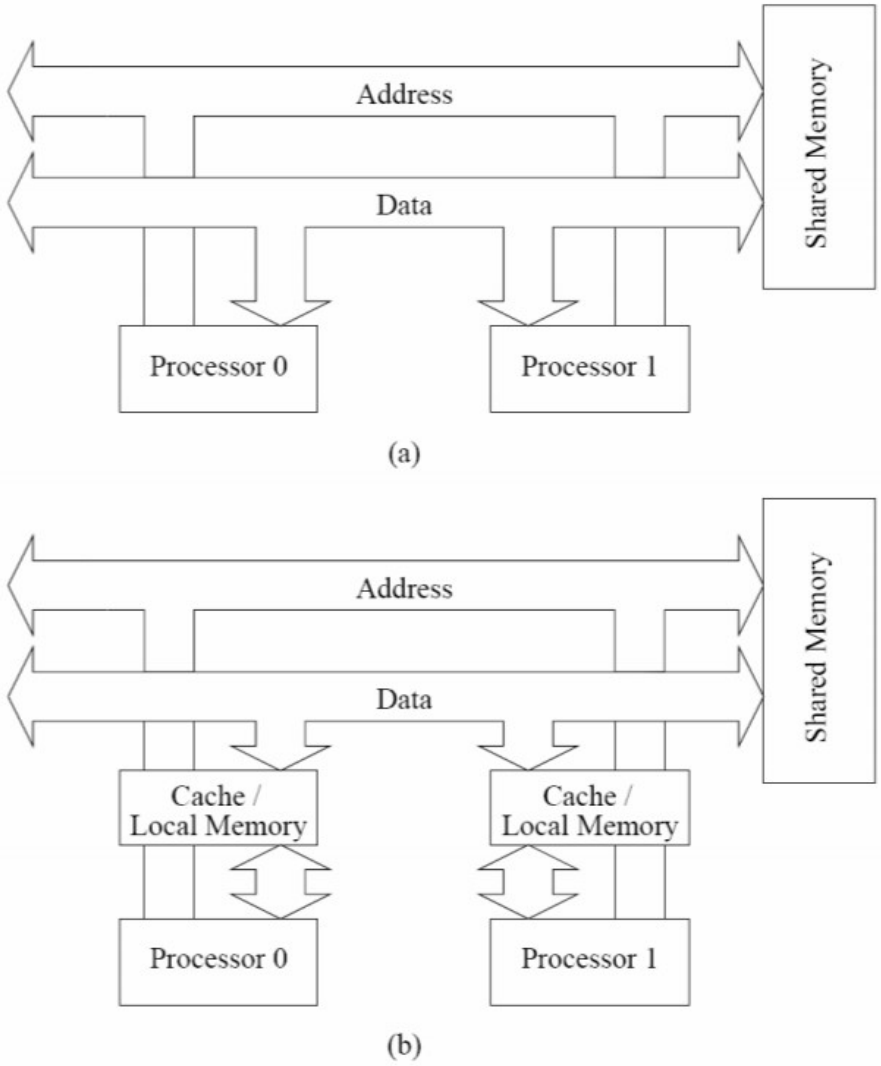
- Lățimea bisecțiunii într-o rețea dinamică se determină prin examinarea mai multor tăieturi care împart rețeaua în două mulțimi egale de noduri.
- Lațimea bisecțiunii este numărul minim de muchii care traversează taietura.
- În figura alăturată:
  - Toate tăieturile sunt traversate de 4 muchii.
  - Lățimea bisecțiunii acestul graf este 4.



Topologiile rețelelor

Rețele bazate pe magistrale (en., bus-based)

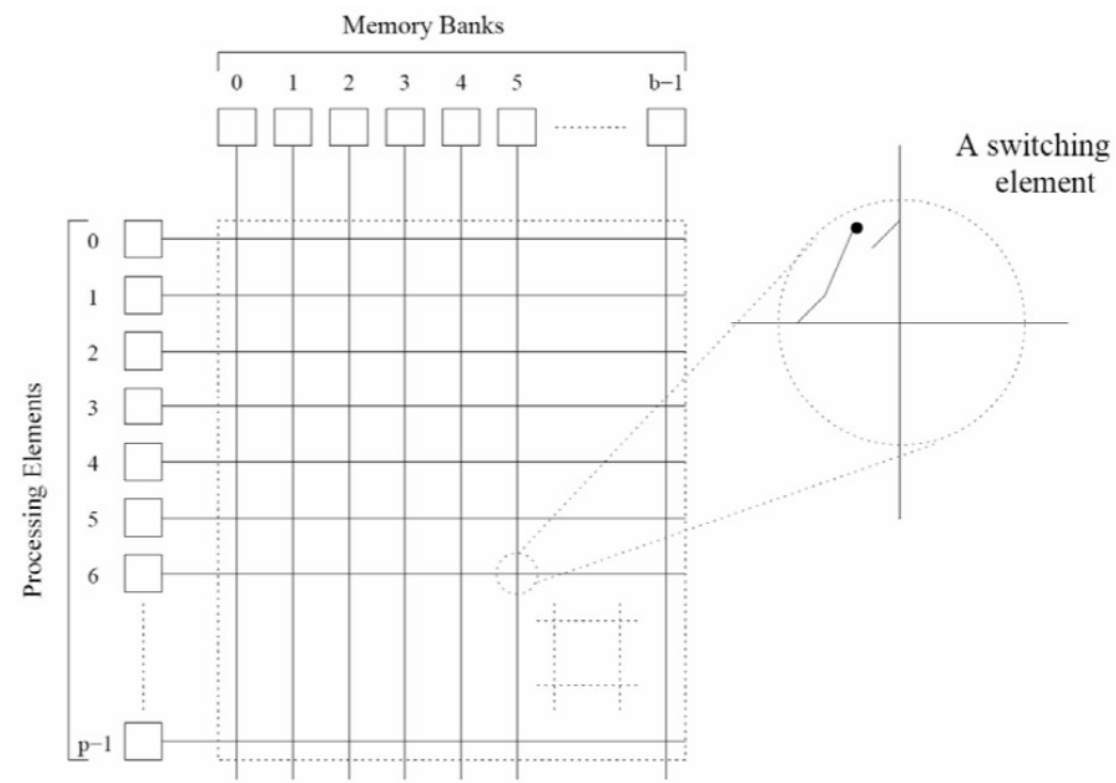
- Partajare minimă.
- Datele sunt transmise tuturor procesoarelor (i.e., broadcast).
- Evaluare:
  - Diametru: O(1)
  - Conectivitate: O(1)
  - Lățimea bisecțiunii: O(1)
  - Costul: O(p)



Topologiile rețelelor

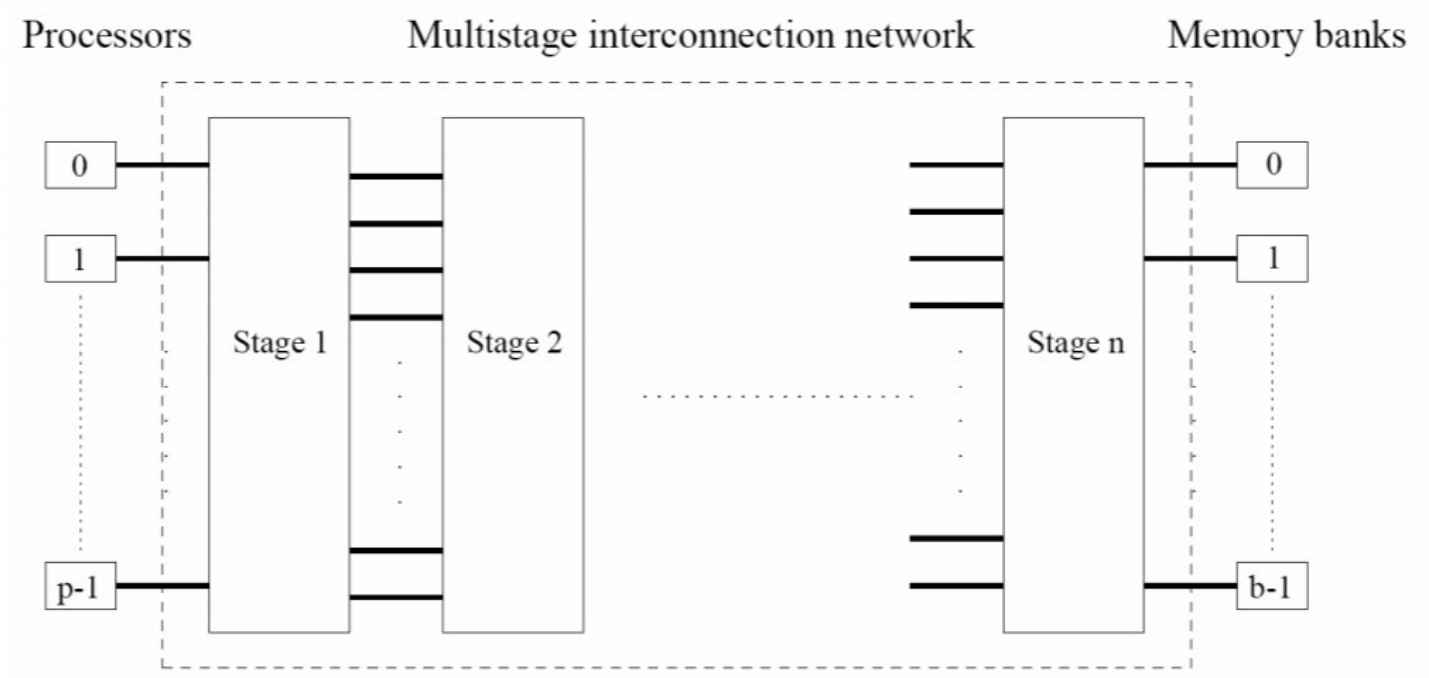
Rețele bazate pe traversare (en., crossbar)

- Folosesc comutatoare
- Permit conexiuni simultane
- Evaluare:
  - Diametru: O(1)
  - Conectivitate: O(1)
  - Lățimea bisecțiunii: O(p)
  - Costul: O(p^2)



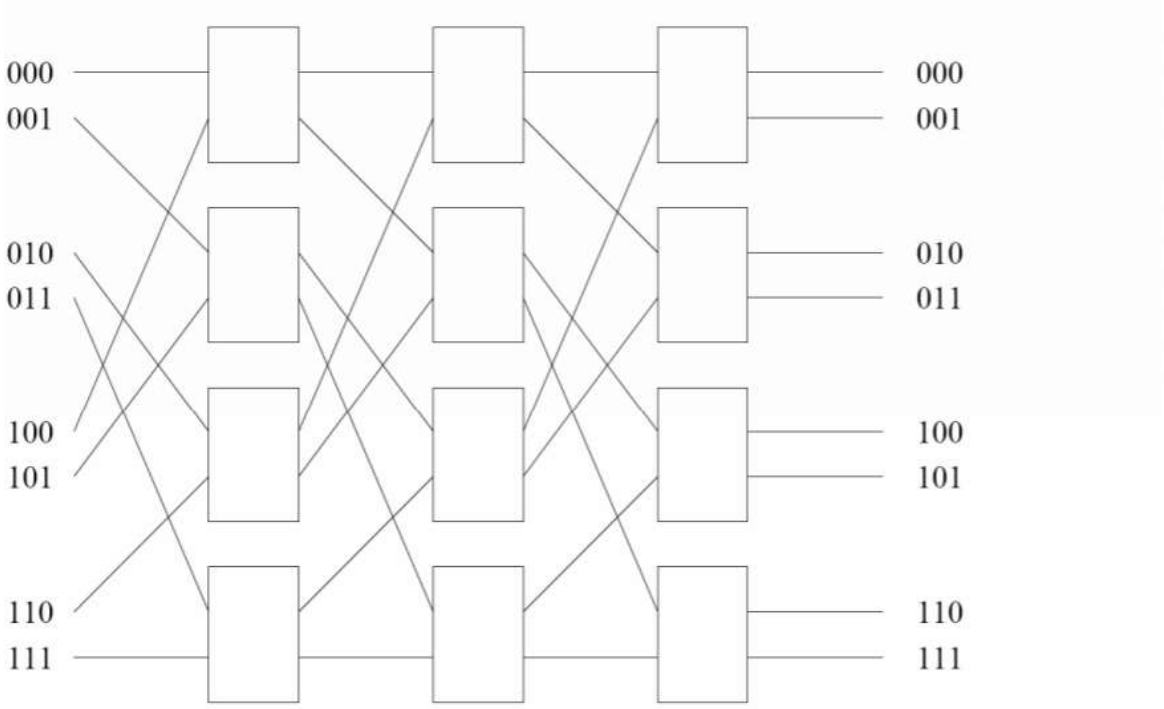
Topologiile rețelelor

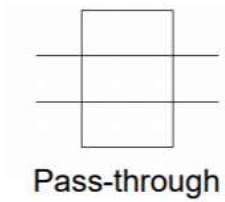
Rețele bazate pe interconectare pe nivele/faze

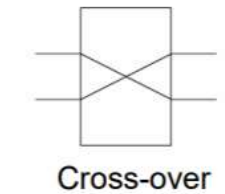


Topologiile rețelelor

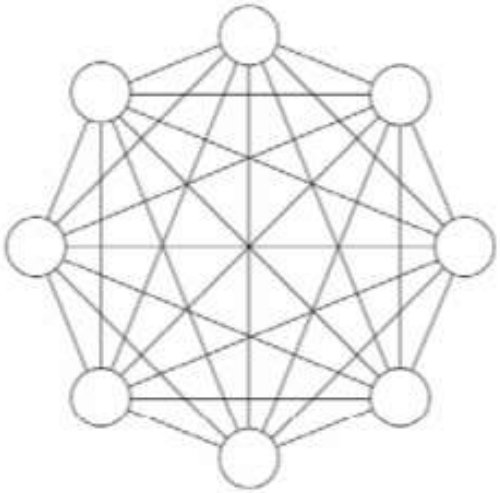
Rețele bazate pe interconectare pe nivele/faze cu comutatoare



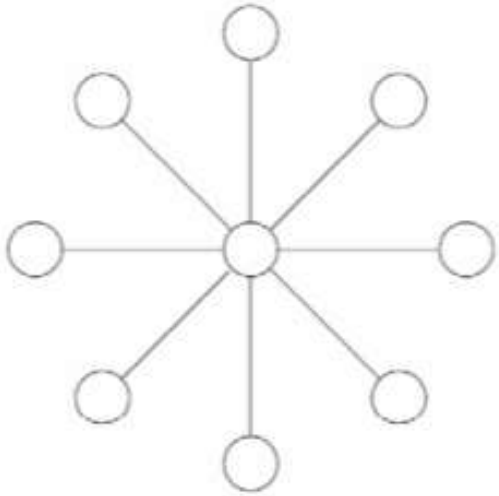




Topologiile rețelelor



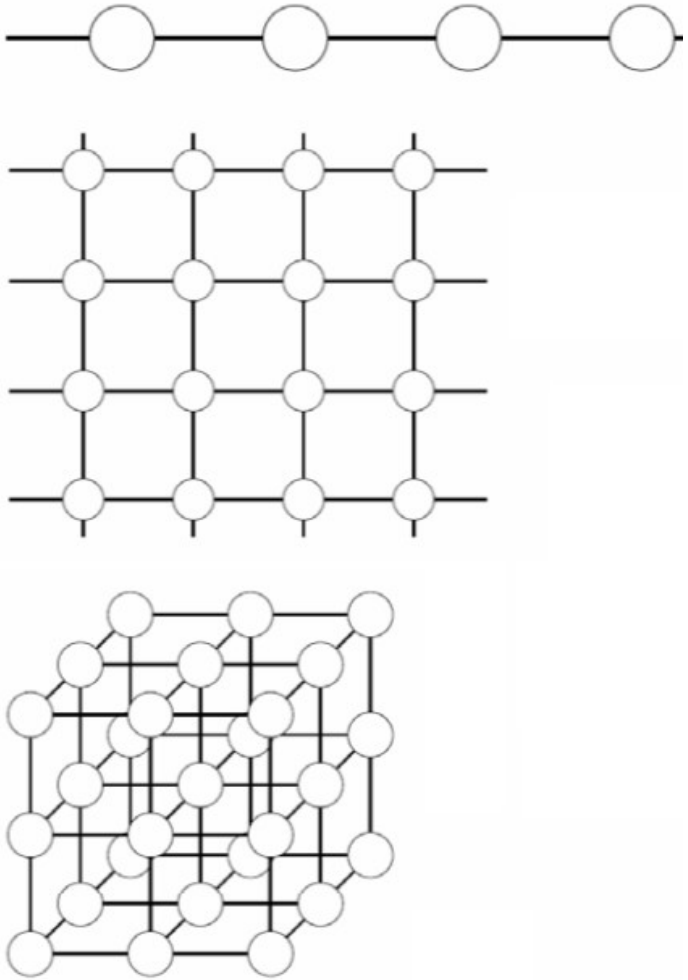
Rețea completă



Rețea stea

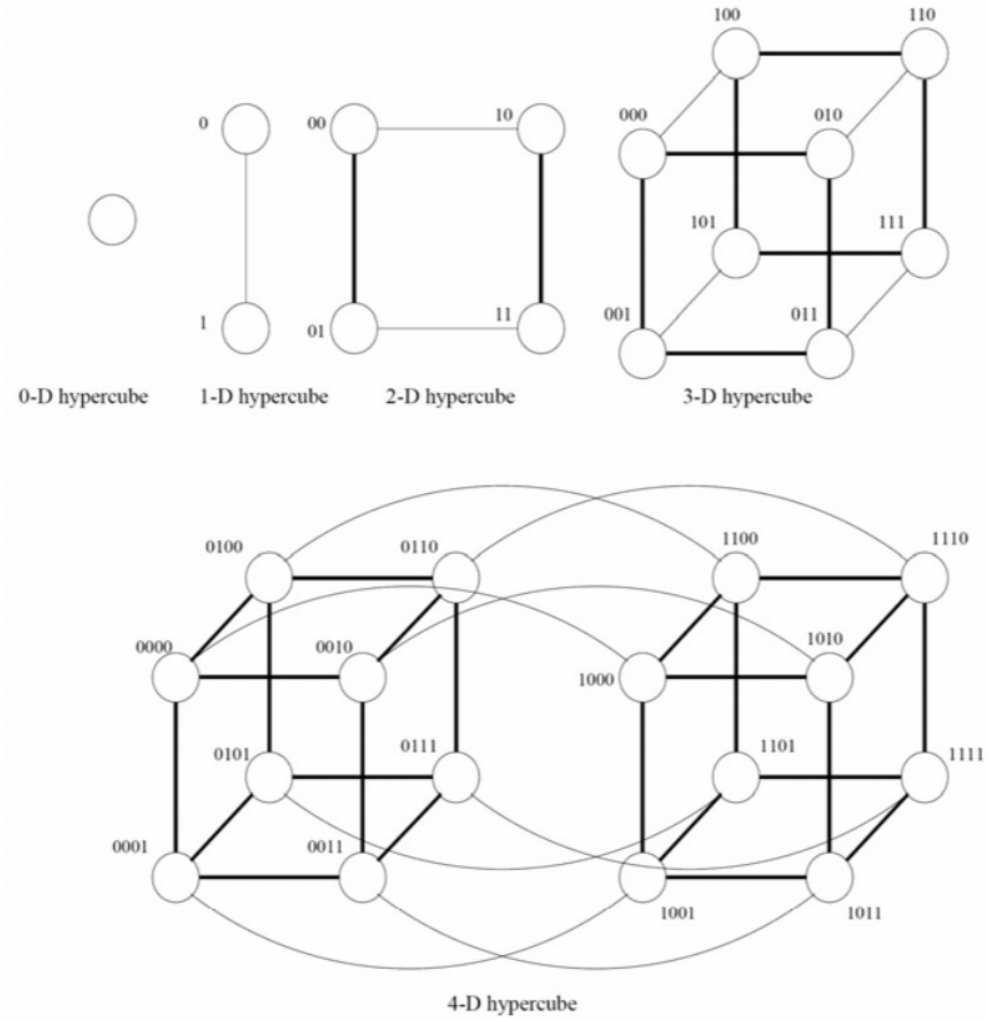
Topologiile rețelelor

- Topologii carteziene:
  - Liniare
  - 2-D
  - 3-D

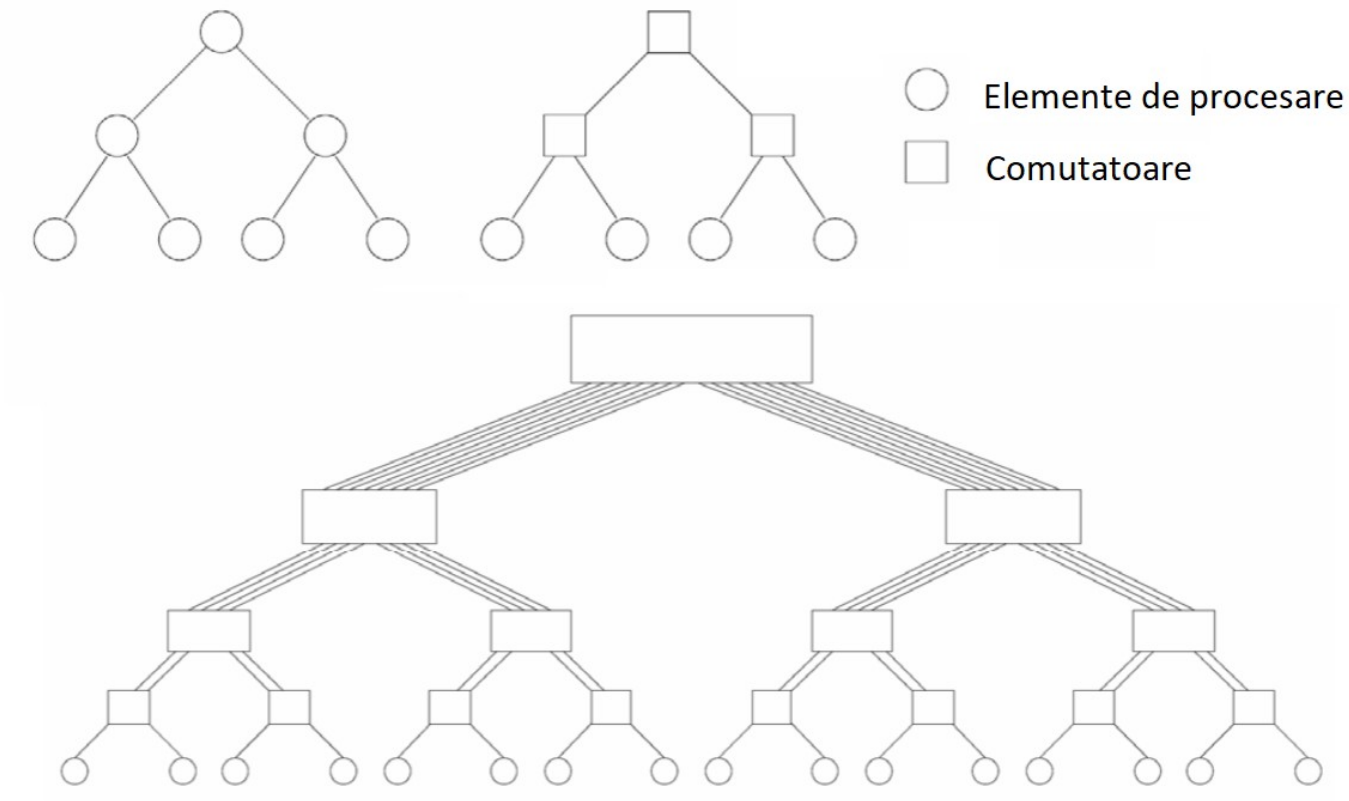


Topologiile rețelelor

- Hipercuburi



Topologiile rețelelor: structuri arborescente

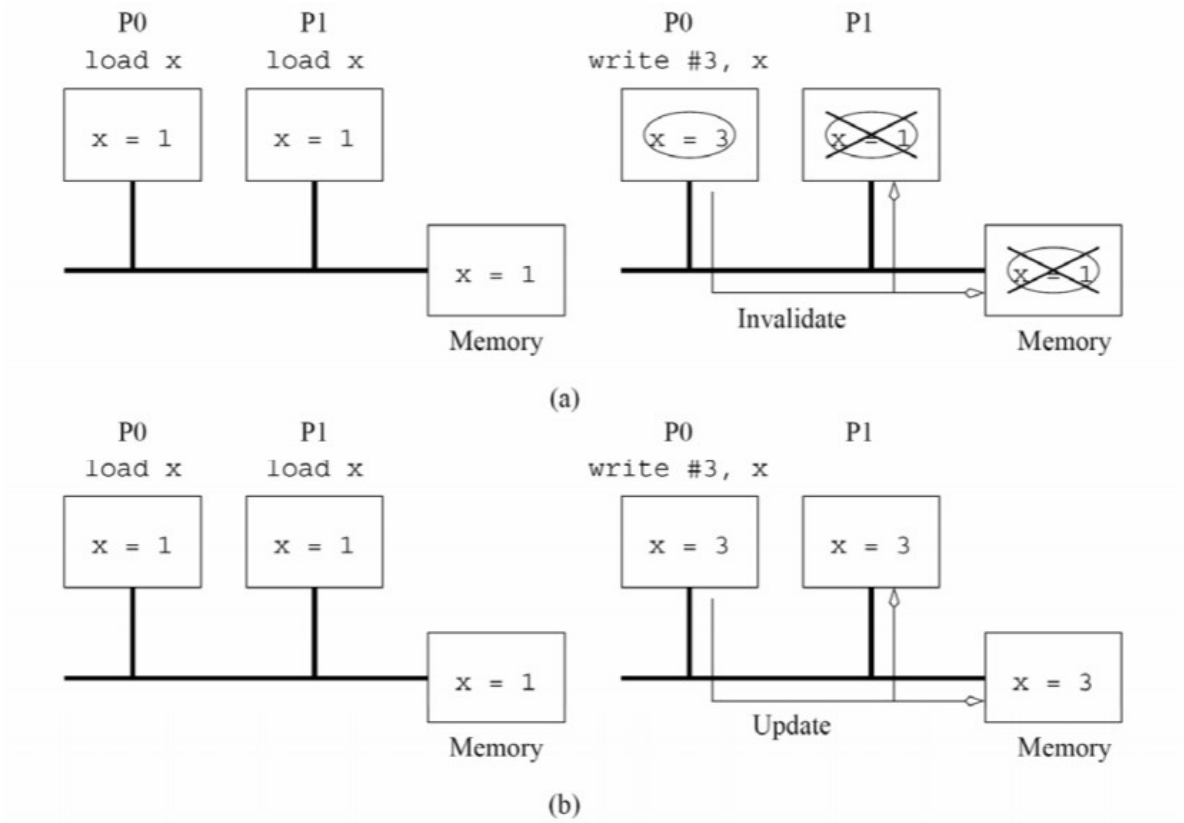


Organizarea fizică

- Coerența cache-urilor în sistemele cu memorie partajată
  - Un anumit nivel de consistență trebuie asigurat atunci când avem mai multe copii ale aceloraşi date.
  - Este necesară pentru a asigura execuția corectă a programului
    - Serializarea datelor
  - Sunt folosite două protocoale pentru a asigura coerența cache-urilor
    - Invalidarea
    - Update-ul

Organizarea fizică

Coerența cache-urilor în sistemele cu memorie partajată: a) invalidarea, b) update-ul



Protocoalele Invalidare/Update

- Metoda folosită depinde de caracteristicile aplicației: frecvența operațiilor de citire/scriere a variabilelor din memoria partajată
- Negocierea (en., trade-off) dintre:
  - Efectuarea update-urilor care introduce nevoie mare de comunicare

  și
  - Funcționarea în gol (en., idling) prin netrimiterea invalidării
- Pot aparea probleme din cauza falsei partajări
- Protocolul de invalidare este mai des utilizat
  - Au fost dezvoltate mai multe abordări pentru menținerea/gestionarea stării datelor partajate.

Costurile de comunicare în sistemele paralele

Sistemele cu transmitere de mesaje

- Costul de comunicare pentru o operație de transfer de date depinde de:
  - **Ts** : timpul de inițializare: adaugarea header-ului, executarea algoritmului de routare, stabilirea conexiunii dintre sursă și destinație
  - **Th** : timpul necesar transferului între două noduri adiacente, adică latența.
  - **Tw** : timpul necesar transferului unui cuvânt, este 1/lățimea canalului de comunicație

Model clasic de comunicare

- Costul trimiterii unui mesaj de dimensiune *m* este

$$T\ com_m = Ts + Tw * m$$

- Reprezintă un model suficient de realist deoarece de obicei **Ts** este mult mai mare decât **Th**, iar pentru cei mai mulți algoritmi ***m * Tw*** este mult mai mare decât ***l * Th***

Mecanisme de routare

- Routarea este algoritmul folosit pentru determinarea căii pe care o va parcurge un mesaj de la sursă la destinație
- Poti fi clasificate astfel:
  - Minimale/optime vs. Suboptimale
  - Deterministe vs. Adaptive
- Dezvoltarea algoritmilor de routare specifici topologiei
- Se are în vedere înglobarea metricilor de calitate
  - Dilatarea = numărul maxim de linii atribuite unei muchii/conexiuni
  - Congestionarea = numărul maxim de muchii atribuite unei singure legături

Accelerarea (en. speedup)

$$S(p) = \frac{\text{Timpul de execuție folosind un procesor (cel mai bun timp secvențial)}}{\text{Timpul de execuție folosind } \boldsymbol{p} \text{ procesoare}} = \frac{T_s}{T_p}$$

S(p) reprezintă creșterea vitezei prin folosirea mai multor procesoare.

Obs: Trebuie folosit cel mai bun algoritm secvențial pe un singur procesor!

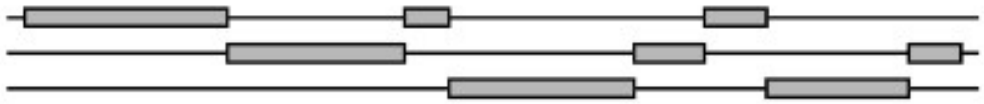
- Varianta (algoritmică si/sau implementarea) paralelă a algoritmului secvențial poate fi (și de obicei este) diferită.

Abordări ale calculului paralel: clasificare

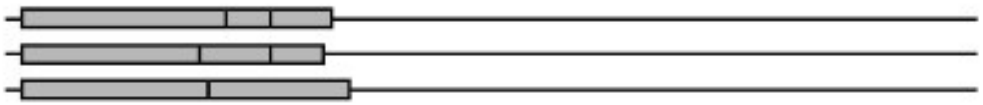
- **Pipelining**: instrucțiunile sunt descompuse în operații elementare/atomice; astfel, mai multe operații se pot executa simultan.
- **Paralelism funcțional**: permite unor sisteme (ex., procesoare) independente să execute instrucțiuni/funcții distincte/specializate.
- **Paralelism vectorial**: permite unor sisteme (ex., procesoare) *identice* să execute *aceeași* instrucțiune/funcție pe date diferite în cadrul aceleiași logici de control.
- **Multi-procesare**: mai multe procesoare *strâns cuplate* execută instrucțiuni independente și comunică prin *memorie partajată*.
- **Multi-calcul**: mai multe procesoare *strâns cuplate* execută instrucțiuni indeendente și comunică prin *mesaje*.

Concurență vs. Paralelism

- Concurența: situația în care se află un sistem în care mai multe task-uri sunt ***logic*** active la un moment dat.
- Paralelismul: situația în care se află un sistem în care mai multe task-uri sunt ***fizic*** active la un moment dat.



Concurrent, non-parallel execution



Concurrent, parallel execution

Programele paralele includ *Programele concurente*

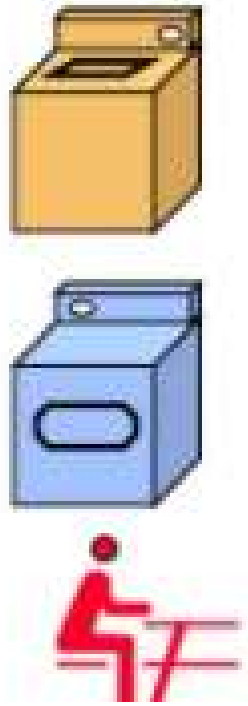
Concurență vs. Paralelism

- **Aplicație concurentă** = aplicație în care task-urile se execută simultan prin semantica aplicației. Aceasta este asigurată prin proiectare adecvată a soluției la o problemă care este intrinsec concurentă.

- **Aplicație paralelă** = Aplicație în care task-urile se execută efectiv simultan astfel încât timpul de rulare să scadă. Problema rezolvată nu trebuie neaparat să fie intrinsec concurentă, ea se poate exprima în mod secvențial.

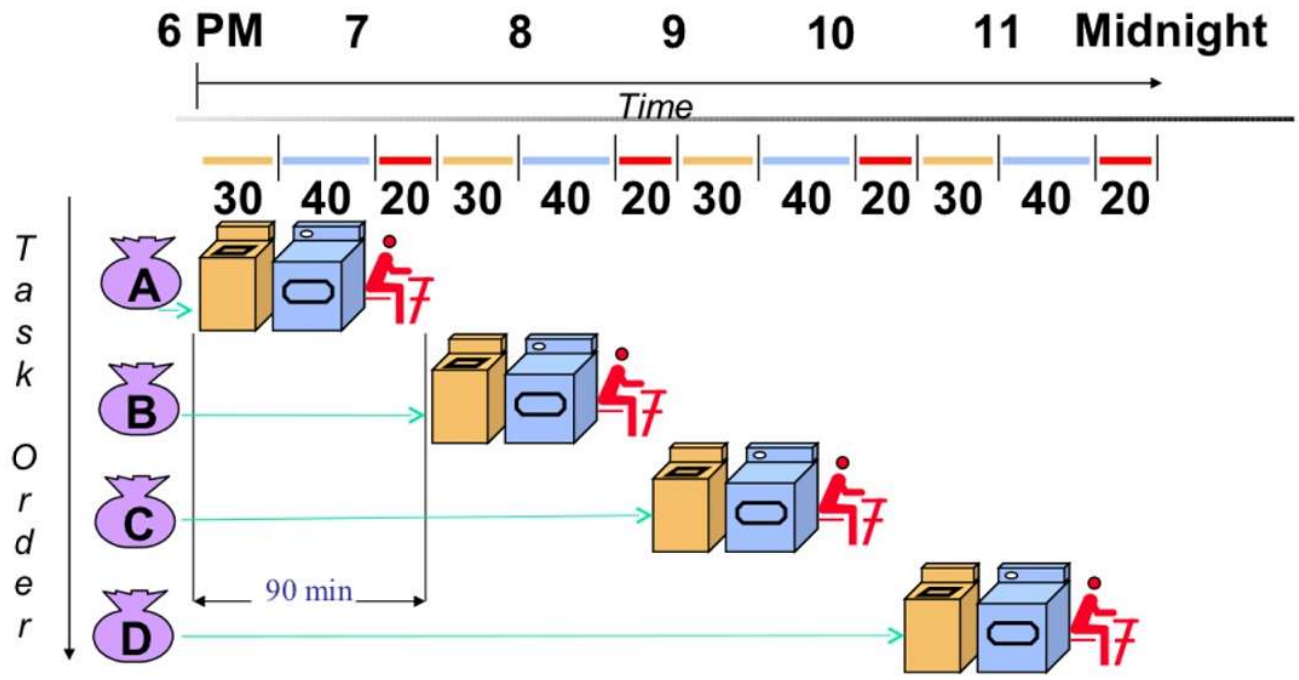
Pipelining: Exemplu

- Avem o spalătorie care are o masină de spălat, una de uscat si un angajat.
- Timpul total de spălare pentru un coș de rufe este 90 de minute:
  - Spălare: 30 de minute
  - Uscare: 40 de minute
  - Împachetarea efectuată de angajat: 20 de minute



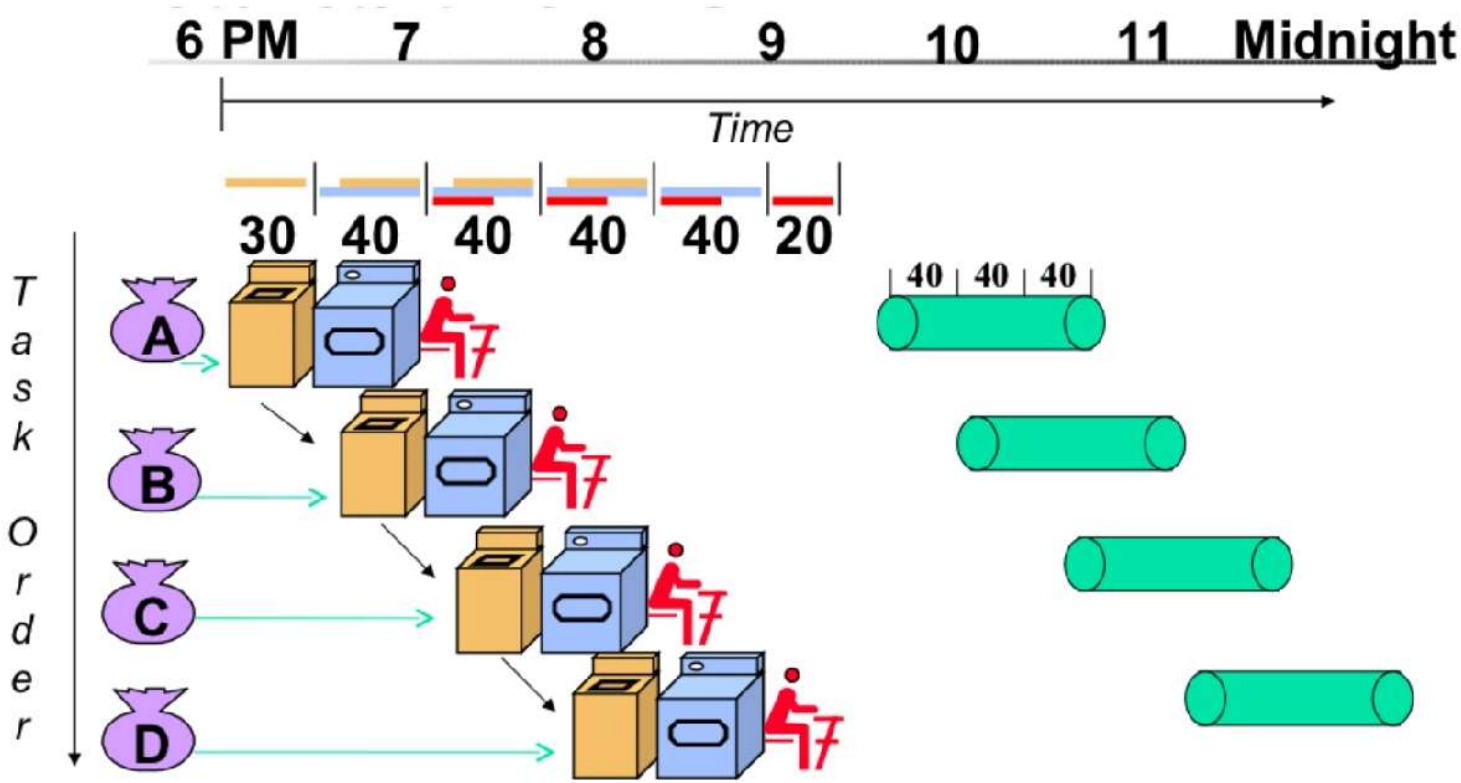
Pipelining: Exemplu: Spălarea secvențială

- Angajatul efectuează impachetarea o dată la 90 de minute. Astfel, el nu va începe o nouă impachetare decât dupa ce un ciclu de spalare-uscare s-a terminat.
- Execuția acestui proces secvențial necesită 6 ore pentru 4 coșuri de rufe.



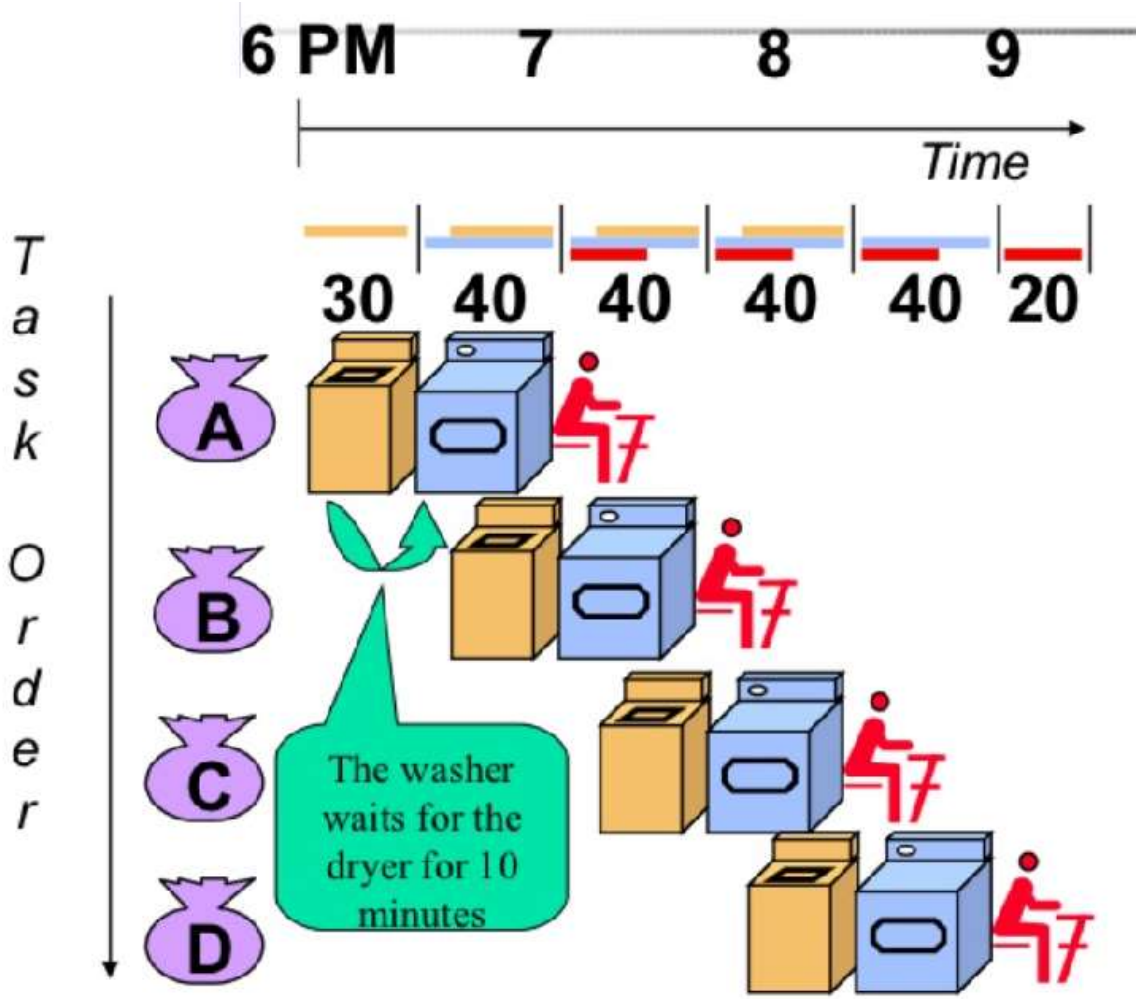
Pipelining: Exemplu: Spălarea eficientă (angajatul împachetează rufele imediat)

- Un **alt** operator realizează încarcarea rufelor în masina de spalat la fiecare 40 de minute?
- Ce se întâmplă daca realizează încărcarea mașinii de spălat imediat după ce aceasta termină spălarea?
- Timpul necesar spălării pentru 4 coșuri devine 3.5 ore. Este acesta timpul optim?



Pipelining: Concluzii

- Mai multe sarcini distincte se desfășoară în paralel.
- Nu se îmbunătățește execuția unei sarcini, ci a intregului proces.
- Limitarea este dată de cel mai lentă sarcină.
- Accelerarea potențială este dată de numarul de stadii ale pipeline-ului.
- Diferența mare între timpul de execuție al sarcinilor conduce la diminuarea accelerării.
- Trebuie să avem în vedere timpul necesar *umplerii* si *golirii* pipeline-ului.

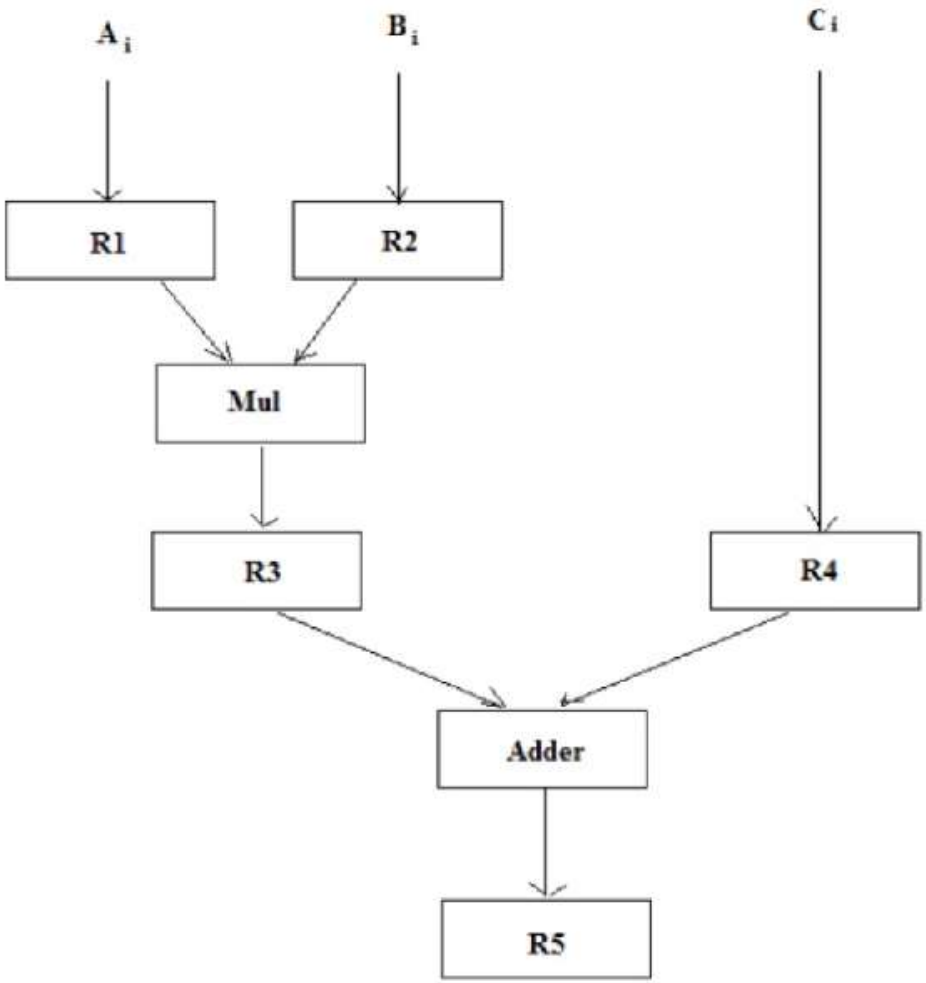


Pipelining: Concluzii

- Metoda pipeline descompune un proces secvențial în segmente/sarcini/task-uri distincte ca tip/procesare.
- Dedică un procesor (ex., hardware) unui anumit tip de task.
- Fiecare task este executat pe procesorul dedicat concurent cu celelalte stak-uri.
- Rezultatele/informația trebuie transmise între procesoare (ex., hardware).
- În general, sarcina totala este împărțită în k segmente/stagii. Toate segmentele necesită aceeași cantitate de timp, adică un ciclu al procesorului. Durata unui ciclu al procesorului este determinată de cel mai încet segment.

Pipelining: Exemplu numeric

- Avem 3 fluxuri de numere A, B si C
- Dorim sa calculăm Ai * Bi + Ci pentru i=1, 2, 3, ... 7
- Definim urmatoarele operații/segmente
  - **R1** <- Ai, **R2** <- Bi
  - **R3** <- R1 * R2, **R4** <-Ci
  - **R5** <- R3 +R4



Pipelining: Exemplu numeric (starea regiștrilor)

Clock Pulse Number	Segment 1		Segment 2		Segment 3
	$R1$	$R2$	$R3$	$R4$	$R5$
1	A_1	B_1	—	—	—
2	A_2	B_2	$A_1 * B_1$	C_1	—
3	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8	—	—	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9	—	—	—	—	$A_7 * B_7 + C_7$

Performanța pipeline-ului

- **N** = dimensiunea problemei (ex., numarul de coșuri cu rufe, dimensiunea fluxurilor de numere A, B si C)
- **K** = numarul de segmente din pipeline (ex., 3 la spălătorie [spălare, uscare, împachetare], 3 la exemplul numeric [R1/R2, R3/R4 și R5])
- **T** = timpul necesar unui ciclu al procesorului (este timpul necesar execuției celui mai încet task [40 min la spalatorie], segmentul 2 la exemplul numeric).
- **Tk** = timpul total necesar procesării **Tk = (K + (N-1)) * T**
- **Accelerarea**

$$\textbf{Speedup} = T1/Tk = N*K / (K + (N-1))$$

Ex: Pentru K=3 si N=4 avem nevoie de K cicluri pentru umplerea pipeline-ului, apoi (N-1) rezultate vor fi disponibile la fiecare ciclu, apoi (K+N-1) cicluri vor fi necesare pentru finalizarea sarcinii. Executia secvențială necesită K*N cicluri ale procesorului.

Obs. Pentru N >>> K (N=1 mil de fisiere, K=3 segmente), **Speedup ≈ k**. Majoritatea procesării se realizează în paralel, mai putin umplerea și golirea pipeline-ului.

Modelul de programare cu fire de execuție (POSISX)

- Toată memoria este globală și poate fi accesată de toate firele de execuție (en., threads)
  - Stiva este locală dar poate fi partajată
- Firele POSIX (API/Pthreads)
  - Nivel jos de programare
  - Au apărut ca standard susținut de cele mai multe SO, acest lucru asigură portabilitatea.
  - Oferă funcții care perimit crearea, terminarea și sincronizarea firelor
    - Aceste funcții sunt de nivel jos care nu oferă modalități de nivel înalt pentru partajarea eficientă a datelor
    - Nu există noțiunea de comunicare colectivă ca aceea oferită de MPI.

PThreads

- Crearea si finalizarea firelor
- Primitive de sincronizare
  - Lacăte pentru excludere mutuală (en., mutual exclusion locks)
  - Variabile condiție (en., conditional variables)
  - Atribute obiect

Crearea firelor

```
#include <pthread.h>
int
pthread_create (
    pthread_t    *thread_handle,
    const pthread_attr_t    *attribute,
    void *    (*thread_function)(void *),
    void    *arg);
```

```
int
pthread_join (
    pthread_t    thread,
    void    **ptr);
```

- https://man7.org/linux/man-pages/man3/pthread_create.3.html
- https://man7.org/linux/man-pages/man3/pthread_join.3.html

Primitive de sincronizare

- Accesul la variabilele partajate trebuie controlat pentru a înlătura situațiile de concurență (en., race conditions) astfel încât să se asigure rularea *semantic/logic* secvențială și în consecință determinismul programului.

/*fiecrae fir încearcă să modifice variabila *best_cost astfel*/

if (my_cost < best_cost)

best_cost = my_cost;

- Presupunem că avem două fire de execuție
- Valoarea inițială pentru *best_cost* este 100.
- Valorile pentru *my_cost* sunt 50 și 75 în firele t1 și t2, respectiv.
- Dacă ambele fire execută *if*-ul concurent atunci amandouă vor executa ramura *then.* În funcție de care fir se va executa primul valoarea *best_cost* va fi fie 50, fie 75.
- Problemă importantă:
  - Natura nedeterministă a rezultatului! Rulări diferite pe acelasi input produc rezultate diferite! ☹

Lacăte (en. locks) pentru excludere mutuală

- O variabila specială denumită *mutex* poate fi folosită pentru a gestiona secțiunile critice dintr-un program
  - Un fir de execuție dobândește lacătul înainte să execute secțiunea critică și îl eliberează după ce execută secțiunea critică.
  - Dacă lacătul este deja în proprietatea altui fir, atunci firul se blochează până când lacatul este eliberat.
- Lacătul reprezintă modalitatea prin care se realizează rularea secvențială, astfel încât utilizarea unui număr prea mare conduce la degradarea performanței.

Lacăte pentru excludere mutuală (cont.)

```
int
pthread_mutex_lock (
    pthread_mutex_t   *mutex_lock);
```

Funcția *pthread_mutex_lock* încearcă să dobândească lacătul *mutex_lock*. Daca acesta este în proprietatea altui fie (este închis) atunci firul apelant se blochează. În caz contrar, firul apelant blochează/închide lacătul si apoi rulează. Rularea cu succes returnează valoarea 0. Alte valori indică erori, de exemplu blocarea (en. deadlock).

```
int
pthread_mutex_unlock (
    pthread_mutex_t *mutex_lock);
```

Funcția *pthread_mutex_unlock* deschide lacătul astfel încat unul dintre firele blocate care așteaptă va intra în secțiunea critică prin închiderea lacătului. Determinarea firului care inchide lacătul și rulează se face cu ajutorul unei politici de planificare (en. schedule policy).

Lacăte pentru excludere mutuală (cont.)

```
int
pthread_mutex_init (
    pthread_mutex_t   *mutex_lock,
    const pthread_mutexattr_t   *lock_attr);
```

Funcția *pthread_mutex_init* inițializează *mutex_lock* în starea deschis. Atributele sunt specificate prin parametrul *lock_attr*.

```
int
pthread_mutex_trylock (
    pthread_mutex_t *mutex_lock);
```

Funcția *pthread_mutex_trylock* încearcă să închidă lacătul *mutex_lock.* Dacă încercarea este reușită atunci funcția returnează 0. Daca lacătul este deja închis de alt fir atunci apelul returnează ***EBUSY***, ceea ce permite firului să execute alt cod. Această funcție este mai rapidă decât *pthread_mutex_lock.*

Variabile condiție (en., conditional variables)

- Se aplică principiile de sincronizare de la cozile de așteptare
  - În funcție de rezultatul unei anumite condiții un fir se poate atașa la o anumită coadă de așteptare
  - Mai târziu, alt fir care schimbă rezultatul condiției va *trezi* toate firele care așteaptă astfel încât acestea pot lua în calcul continuarea rulării.
- Variabilele condiție sunt întotdeauna asociate cu un lacăt.

API pentru variabilele condiție

```
int pthread_cond_wait(pthread_cond_t *cond,
    pthread_mutex_t *mutex);
```

```
int pthread_cond_broadcast(pthread_cond_t *cond);
```

```
int pthread_cond_signal(pthread_cond_t *cond);
```

```
int pthread_cond_timedwait(pthread_cond_t *cond,
    pthread_mutex_t *mutex,
    const struct timespec *abstime);
```

```
int pthread_cond_init(pthread_cond_t *cond,
    const pthread_condattr_t *attr);
int pthread_cond_destroy(pthread_cond_t *cond);
```

Obiecte atribut

- Diverse atribute pot fi asociate cu fire, lacăte și variabile condiție.
- Atributele gestionate de fire sunt:
  - Parametrii politicii de planificare (en., schedule policy)
  - Dimensiunea stivei
  - Diverse stări în cadrul programului
- Atributele lacătelor sunt:
  - Normale
    - Numai un fir îl poate deține pentru a-l închide
    - Daca un fir incearcă să îl dețină de 2 ori atunci programul se blocheaza (en., deadlock).
  - Recursive
    - Un fir poate închide un lacăt de mai multe ori
    - Fiecare închidere incrementează un contor, si fiecare deschidere descrește acel contor
    - Un fir poate închide un lacăt numai daca are contorul zero.
  - Verificarea erorilor
    - Încercarea unui fir de a închide un lacăt pe care îl deține conduce la eroare.